

ZAP-FIX: A Zero-Copy, Post-Quantum Transport for the Financial Information eXchange Protocol

ZAP Protocol Authors
zap-protocol.io

May 6, 2026

Abstract

The Financial Information eXchange (FIX) protocol is the dominant session-layer protocol for institutional equities, futures, and derivatives order flow. Its standard transport binding (raw TCP, or TCP with TLS 1.3) was designed in an era when (a) bandwidth was the binding constraint, (b) nanosecond-scale latency was a curiosity rather than a regulatory expectation, and (c) cryptographically relevant quantum adversaries were a long-horizon concern. None of these conditions hold for institutional venues today. We present *ZAP-FIX*, a transport binding for FIX 4.4 and FIX 5.0 SP2 that preserves the FIX session layer in full—SequenceNumber discipline, Logon/Logout, GapFill, ResendRequest, Heartbeat—while replacing the underlying TCP/TLS stream with the ZAP zero-copy, mutually authenticated, post-quantum frame protocol. We give a formal latency model, a compatibility analysis, a resilience model, and a compliance mapping to MiFID II RTS 25 timestamping. Our analysis shows that ZAP-FIX delivers steady-state per-message latency strictly less than or equal to FIX-over-TLS, with the gap dominated by the elimination of SOH (0x01) text scanning in favour of zero-copy schema fields, and that session-resumption latency is bounded by a single ZAP handshake even after pod migration or network partition—a regime in which TCP+TLS today requires connection re-establishment plus session-state reconstruction.

1 Introduction

The Financial Information eXchange (FIX) protocol, in versions 4.2, 4.4, and 5.0 SP2, is the de facto session protocol for institutional order flow across cash equities, listed futures, options, and a large fraction of OTC derivatives [7, 9]. Conservative public estimates put the share of institutional equity order messages carried over a FIX session at well above ninety percent. The protocol layers cleanly over a reliable byte-stream transport: implementations overwhelmingly choose TCP, with TLS 1.3 layered in for venues subject to confidentiality or integrity regulation [4, 15].

Two pressures are now reshaping the transport choice. First, deterministic tail latency is an explicit best-execution obligation under MiFID II RTS 25, which requires microsecond-granular timestamps for many asset classes [5]. Co-located institutional clients today operate inside a round-trip latency budget of approximately 50–200 μ s from order entry to acknowledgement, with tail latencies dominated not by steady-state per-message cost but by TLS handshake re-establishment and FIX session-state reconstruction following a transient disconnection [11, 12]. Second, the prospect of a cryptographically relevant quantum adversary has motivated regulators and venue operators to require, or at least to permit, post-quantum confidentiality for stored and in-flight order flow, under a “harvest-now, decrypt-later” threat model [2, 13].

Contribution. We define *ZAP-FIX*: a transport binding that preserves the FIX session protocol as specified by FIX Protocol Limited and substitutes only the byte-stream layer. Concretely:

- Each FIX administrative or application message becomes exactly one ZAP frame. There is no in-flight chunking and no aggregation of multiple FIX messages into a single frame, preserving message-boundary semantics.
- The SOH-delimited `tag=value` body is wrapped in a Cap’n Proto envelope; a small, fixed set of routing fields (`SenderCompID`, `TargetCompID`, `MsgSeqNum`, `MsgType`) is lifted into typed envelope slots so that the gateway can dispatch without scanning the FIX body.
- Session establishment uses the ZAP handshake, which provides mutual authentication and X-Wing hybrid post-quantum key agreement [1, 14] in a single round trip.
- SequenceNumber discipline is preserved exactly: ZAP frame ordering on a session is constrained to match FIX `MsgSeqNum` monotonicity, so `ResendRequest` and `GapFill` semantics remain intact.
- An optional *ZAP-FIX BinaryEncoding* mode bit-packs frequent typed fields (`Symbol`, `Price`, `OrderQty`) directly into the envelope, reducing on-wire bytes by 30–50% relative to SOH text and eliminating the corresponding format/parse cost. This mode is conceptually adjacent to FIX SBE [10] and FIX FAST [8], but it inherits the ZAP framing and cryptographic guarantees rather than layering separately.

We also specify a transparent ZAP-FIX gateway, which terminates legacy TCP/TLS-FIX on the front side and speaks ZAP-FIX on the back side, so that institutional clients carrying long-lived connectivity contracts and certifications can adopt ZAP-FIX incrementally without endpoint migration.

The remainder of the paper is organised as follows. Section 2 recaps the FIX session layer. Section 3 specifies the ZAP-FIX construction. Section 4 gives the latency analysis. Section 5 describes the resilience model. Section 6 maps the construction to MiFID II RTS 25 and FINRA OATS audit obligations [3, 5, 6]. Section 7 discusses compatibility with deployed FIX infrastructure. Section 8 surveys related work.

2 FIX Session Layer Recap

We adopt the standard FIX session-layer abstractions [7, 9].

A *session* is a bidirectional, sequenced exchange of FIX messages between two parties identified by `SenderCompID` and `TargetCompID`. Each direction maintains a strictly increasing `MsgSeqNum`. The protocol is intentionally stream-oriented: the underlying transport is required to deliver bytes in order and to surface gaps, but is permitted to coalesce or fragment messages at byte granularity. The session layer compensates with a small set of administrative messages.

Logon (A). Initiates a session, negotiates encryption (historically; in modern deployments `EncryptMethod` = 0 and confidentiality is provided by the transport), heartbeat interval, and optional sequence-number reset.

Heartbeat (0) and TestRequest (1). Liveness probes; absence of either over an interval triggers session termination.

ResendRequest (2) and SequenceReset (4). The receiver, on detection of a gap (`MsgSeqNum` greater than expected), issues `ResendRequest` for the missing range; the sender replays application messages and may emit a `SequenceReset`–`GapFill` for administrative messages it declines to replay.

Logout (5). Graceful termination.

The transport contract that the FIX session layer requires is therefore quite narrow: in-order, reliable, byte-oriented delivery within a single connection lifetime, with explicit failure (the connection is torn down) when delivery cannot be guaranteed. Any transport satisfying this contract is admissible. ZAP, restricted to a single-session ordered binding, satisfies it; we make this precise in Section 3.

3 ZAP-FIX Construction

3.1 Frame Mapping

Let M be a FIX message with header fields $h = (\text{SenderCompID}, \text{TargetCompID}, \text{MsgSeqNum}, \text{MsgType})$ and body b (the `tag=value` fields excluding h , with the SOH-delimited textual encoding $\sigma(M)$).

The ZAP-FIX frame for M is the Cap'n Proto envelope

$$F(M) = \langle \text{hdr}(h), \text{body}(M) \rangle$$

where $\text{hdr}(h)$ is a fixed-layout typed struct exposing the four routing fields, and $\text{body}(M)$ is one of the following two encodings, negotiated at session Logon.

- **Text mode.** $\text{body}(M) = \sigma(M)$. The SOH-delimited canonical FIX body is carried verbatim as a single byte slice. Conformance to the FIX specification is bit-for-bit on the body; the envelope is metadata only.
- **BinaryEncoding mode.** A small dictionary of typed fields (`Symbol`, `Price`, `OrderQty`, `Side`, `OrdType`, `TimeInForce`, `TransactTime`, `ClOrdID`) is bit-packed in the envelope. Remaining fields fall through to a residual SOH body. Decoders MUST be capable of reconstructing the canonical SOH form from the envelope. This mirrors the intent of FIX SBE [10] but does not require a separate transport binding.

One-message-per-frame invariant. For every FIX message M , exactly one ZAP frame $F(M)$ is emitted on the session, and ZAP frames are not coalesced. This is the only deviation from a faithful byte-stream transport, and it strengthens, rather than weakens, the FIX session contract: gaps are detectable at frame granularity rather than at byte granularity.

3.2 Session Establishment

ZAP-FIX session establishment is a strict superset of the FIX Logon exchange:

1. **ZAP handshake.** A single round trip exchanges static and ephemeral X25519 public keys, X-Wing encapsulations [1, 14], and a transcript hash. On success, both parties hold a shared traffic secret and a peer identity bound to a static long-term key.
2. **FIX Logon (A).** The first FIX frame carries Logon. `EncryptMethod` is fixed to 0 (transport provides confidentiality and integrity). `HeartBtInt`, `ResetSeqNumFlag`, and `NextExpectedMsgSeqNum` retain their FIX semantics.

The handshake binds the FIX `SenderCompID` to the ZAP identity. The gateway rejects the Logon if the asserted `CompID` does not appear in the entitlement list authorised for the ZAP identity, defending against a class of `CompID`-spoofing attacks that pure TCP/TLS-FIX cannot detect at the transport layer.

3.3 Sequence-Number Discipline

The ZAP-FIX session maintains, for each direction, the FIX invariant

$$\text{MsgSeqNum}(F_{i+1}) = \text{MsgSeqNum}(F_i) + 1. \quad (1)$$

Because frames are not coalesced and ZAP delivery is in-order on a session, the receiver detects a gap by inspecting the envelope header alone, without parsing the SOH body. This is the basis for the parse-cost advantage of Section 4 and for faster `ResendRequest` of Section 5.

4 Latency Analysis

We decompose the per-message round-trip latency between two co-located peers into:

$$T_{\text{rtt}} = T_{\text{ser}} + T_{\text{tx}} + T_{\text{net}} + T_{\text{rx}} + T_{\text{parse}} + T_{\text{app}} + T_{\text{rev}} \quad (2)$$

where T_{ser} is sender-side serialisation, T_{tx} kernel transmit, T_{net} wire and switch fabric, T_{rx} kernel receive, T_{parse} receiver-side parse, T_{app} application processing (matching, risk, etc.), and T_{rev} the reverse-path contributions.

The transport choice fixes T_{ser} , T_{parse} , and partially T_{net} (via on-wire byte count). T_{app} is unaffected.

4.1 Steady-State Per-Message Cost

Serialisation. For FIX-over-TCP/TLS, T_{ser} is dominated by formatting typed fields into ASCII and inserting SOH separators; in optimised implementations this lands in the 1–5 μs range per message. For ZAP-FIX text mode, the cost is the same plus a constant Cap’n Proto envelope (< 100 ns on contemporary hardware). For ZAP-FIX BinaryEncoding mode, formatting of the hot fields is replaced by bit-packing of pre-typed values (approximately 50–200 ns for the hot field set).

Parse. For FIX-over-TCP/TLS, T_{parse} is dominated by an SOH scan and `tag=value` tokenisation; routing fields are reachable only after the scan reaches them. For ZAP-FIX, routing fields are zero-copy reads from the envelope; an end-to-end FIX session that only requires the routing fields plus a handful of hot fields can avoid touching the SOH body altogether.

On-wire bytes. A typical `NewOrderSingle` in SOH text is in the range 180–280 bytes. ZAP-FIX BinaryEncoding mode reduces this to 110–170 bytes for the same field set. Within a single 1500-byte MTU, T_{net} is unaffected; over multi-segment or constrained fabrics, the byte reduction translates linearly.

We make this precise in Theorem 1 of the companion proof document.

4.2 Handshake and Resumption

A TLS 1.3 handshake completes in one round trip in the 0-RTT or 1-RTT regime [15]. The ZAP handshake also completes in one round trip and yields the same forward-secrecy and authentication properties, plus post-quantum KEM hybridisation [1]. In steady state the handshake cost is amortised; the regime in which it dominates is connection re-establishment, which we treat in Section 5.

5 Resilience

A FIX session over TCP+TLS that loses its underlying TCP connection must:

1. Re-establish a TCP connection (one round trip plus retransmit backoff if loss was the cause).
2. Re-establish a TLS session (one round trip; or 0-RTT resumption with replay-window caveats).
3. Reissue `Logon`.
4. Detect the gap by comparing `MsgSeqNum` expectations.
5. Issue `ResendRequest` and replay the missing range.

Empirically the dominant cost in steps 4–5 is the byte-stream scan of the receive buffer to locate FIX message boundaries, since the TCP byte-stream surface does not expose them.

ZAP-FIX collapses this:

1. One ZAP handshake re-establishes the secure channel.
2. `Logon` reasserts the session.
3. Gap detection is a single integer comparison on the next envelope header.
4. `ResendRequest` is parameterised by frame indices, and replay is read from a frame-addressable durable buffer.

We bound this in Theorem 2 of the companion proof document.

6 Compliance and Audit

MiFID II RTS 25. RTS 25 mandates clock synchronisation and timestamp granularity appropriate to the asset class; for HFT-eligible flow the threshold is $1\ \mu\text{s}$ with a divergence from UTC bounded by $100\ \mu\text{s}$ [5]. ZAP-FIX frames carry a transport timestamp populated from a hardware PTP/IEEE-1588-disciplined clock, signed inside the frame’s authenticated additional data. The signed transport timestamp is in addition to the FIX `TransactTime` (tag 60) carried in the body; the two are independently auditable.

FINRA OATS, ATS-N, CAT. The FIX session-layer artefacts that these regimes require (`ClOrdID`, `OrigClOrdID`, event timestamps, `ExecID`) [3, 6] are unchanged by ZAP-FIX; the binding adds, rather than replaces, audit signal. Crucially, the ZAP frame’s authenticated transcript is itself an admissible audit artefact: the absence of a frame in the durable log implies the absence of the corresponding FIX message at the session layer, with cryptographic strength.

Post-quantum confidentiality. For asset classes whose order intent retains economic value over multi-year horizons (some primary issuance, certain block trades, strategic accumulation programmes), harvest-now-decrypt-later is a realistic threat. ZAP-FIX inherits the X-Wing hybrid KEM [1] of the underlying ZAP transport; FIX-over-TLS does not, in any deployed profile.

7 Compatibility

ZAP-FIX coexists with deployed FIX infrastructure through a transparent translation gateway. The gateway terminates a TCP/TLS-FIX session on its public face and a ZAP-FIX session on its private face. Because the FIX session layer is unchanged, the gateway is a stateful but stateless-with-respect-to-application-semantics relay: it forwards Logon, Logout, Heartbeat, and SequenceReset transparently, and translates between the SOH byte stream and the ZAP envelope.

This means an institutional client whose order-management system is certified against a specific FIX dialect over TCP+TLS need not recertify or modify its endpoint. Migration to ZAP-FIX endpoints can proceed at the operator’s pace.

8 Related Work

FIX FAST [8] compresses FIX market-data feeds via implicit field presence and stop-bit encodings; it is widely deployed for multicast market data but uncommon for order entry. FIX SBE (Simple Binary Encoding) [10] replaces SOH text with a fixed-layout binary encoding; it presupposes a transport binding and is most often layered over TCP. ZAP-FIX BinaryEncoding mode is philosophically closest to SBE but inherits transport-layer security and post-quantum guarantees from ZAP.

FIX-over-WebSocket extensions exist for browser-side market data and retail-adjacent order entry; they trade efficiency for firewall traversal. They are not used in colocated institutional channels and are not a competitor in the regime addressed here.

The HFT-latency literature gives detailed budgets and tail-latency analyses against which our model can be calibrated [11,12,16]. Colocated venue architectures from Refinitiv, Bloomberg, and SFTI universally expose TCP+TLS-FIX as the institutional binding; ZAP-FIX is intended as a forward-compatible upgrade path.

9 Conclusion

ZAP-FIX preserves the FIX session protocol bit-for-bit while substituting a zero-copy, mutually authenticated, post-quantum transport for the TCP+TLS byte stream. The construction is transparent to FIX session-layer logic, recoverable through the existing GapFill/ResendRequest mechanisms, and strictly improves on the steady-state and resumption latency of TCP+TLS-FIX. The companion proof document gives the formal latency model.

References

- [1] M. Barbosa, D. Connolly, J. Diaz, P. Kampanakis, V. Krasnov, and B. Westerbaan. X-Wing: The Hybrid KEM You’ve Been Looking For. IETF Draft `draft-connolly-cfrg-xwing-kem`, 2024.
- [2] E. Barker, L. Chen, and R. Davis. Recommendation for Key-Derivation Methods in Key-Establishment Schemes. NIST Special Publication 800-56C Rev. 2, 2020.
- [3] Consolidated Audit Trail, LLC. CAT Reporting Technical Specifications for Industry Members. <https://www.catnmsplan.com/specifications>, 2024.
- [4] editor Eddy, W. Transmission Control Protocol (TCP). RFC RFC 9293, Internet Engineering Task Force, 2022.

- [5] European Securities and Markets Authority. Commission Delegated Regulation (EU) 2017/574 (RTS 25): Level of accuracy of business clocks. Official Journal of the European Union, L 87/148, 2017.
- [6] Financial Industry Regulatory Authority. Order Audit Trail System (OATS) Reporting Technical Specifications. <https://www.finra.org/filing-reporting/order-audit-trail-system-oats>, 2019. Retired 2021; superseded by CAT but referenced for historical channel-design context.
- [7] FIX Protocol Limited. FIX Protocol Specification, Version 4.4. <https://www.fixtrading.org/standards/fix-4-4/>, 2003. Accessed 2026-05-05.
- [8] FIX Protocol Limited. FIX Adapted for STreaming (FAST) Protocol Specification, Version 1.2. <https://www.fixtrading.org/standards/fast/>, 2009. Accessed 2026-05-05.
- [9] FIX Protocol Limited. FIX 5.0 Service Pack 2 with Errata 20110818. <https://www.fixtrading.org/standards/fix-5-0-sp-2/>, 2011. Accessed 2026-05-05.
- [10] FIX Protocol Limited. Simple Binary Encoding (SBE), Version 2.0. <https://www.fixtrading.org/standards/sbe/>, 2020. Accessed 2026-05-05.
- [11] Joel Hasbrouck and Gideon Saar. Low-latency trading. *Journal of Financial Markets*, 16(4):646–679, 2013.
- [12] Robert Kissell. *The Science of Algorithmic Trading and Portfolio Management*. Academic Press, 2013.
- [13] National Institute of Standards and Technology. Post-Quantum Cryptography Standardization. <https://csrc.nist.gov/projects/post-quantum-cryptography>, 2024.
- [14] NIST. Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203 / ML-KEM). NIST FIPS 203, 2024.
- [15] E. Rescorla. The Transport Layer Security (TLS) Protocol Version 1.3. RFC RFC 8446, Internet Engineering Task Force, 2018.
- [16] Elaine Wah and Michael P. Wellman. Latency arbitrage in fragmented markets: A strategic agent-based analysis. *Algorithmic Finance*, 5(3-4):69–93, 2016.